Informe técnico VectorIA

VectorIA: Asistente Inteligente para la Vigilancia del Dengue en Costa Rica

Colegio Universitario de Cartago (CUC) · Proyecto 2 · Inteligencia Artificial

Integrantes: Mariana Méndez Pérez, Luis Diego Montero Vargas, Nadin Rojas López y Claret Rodríguez Jiménez

Profesor: Osvaldo González Chaves

Repositorio: github.com/Mari27MP/VectorIA

1. Introducción

El dengue es una de las enfermedades más importantes de salud pública en Costa Rica. Entre 2014 y 2024 hubo tres brotes fuertes (2016, 2023 y 2024), y en la peor semana (noviembre de 2023) se reportaron 1,749 casos. El virus lo transmite sobre todo el mosquito *Aedes aegypti*, que se reproduce en agua estancada y se ve favorecido por la lluvia y el calor.

El objetivo de VectorIA es ayudar a las autoridades de salud y a las comunidades con tres tareas:

1. **Identificar** si un mosquito en una foto es un vector del dengue (módulo 1, red convolucional CNN).
2. **Predecir** los casos de dengue de las próximas 4 semanas a partir del clima (módulo 2, red recurrente RNN/LSTM).
3. **Recomendar** la acción preventiva más adecuada (módulo 3, red neuronal densa ANN).

Los tres modelos se unen en una aplicación web hecha con Streamlit y en una API REST hecha con FastAPI. En este informe explicamos cómo diseñamos cada parte, qué resultados obtuvimos, qué limitaciones encontramos y qué recomendamos para mejorar el sistema.

2. Datos utilizados

Casi todos los datos del proyecto son reales. Los casos de dengue vienen de la OPS/PAHO (a través de OpenDengue) y el clima viene de NASA POWER. La única parte con etiquetas generadas por reglas es el dataset de acciones de la ANN.

Datos	Módulo	Tamaño	Tipo
Fotos de mosquitos (*Aedes aegypti*, *Aedes albopictus*, *Culex*)	1 · CNN	4,451 imágenes	Reales
Casos semanales nacionales con clima (vectoria_dengue_nacional_real.csv)	2 · LSTM	574 semanas, 2014 a 2024	Reales

Datos	Módulo	Tamaño	Tipo
Clima semanal de 2025 y 2026 (NASA POWER, descargado por nosotros)	2 · LSTM	96 semanas	Reales
Acciones recomendadas (vectoria_acciones.csv)	3 · ANN	2,060 filas, 2017 a 2024	Base real, etiquetas por reglas
Casos anuales por cantón (vectoria_casos_cantonal_anual_real.csv)	App · mapa de calor	72 cantones, 2013 a 2018	Reales

Las fotos se tomaron de los datasets públicos que sugiere el proyecto (IEEE DataPort, Mendeley, Mosquito Alert y Kaggle). No se subieron al repositorio por su tamaño.

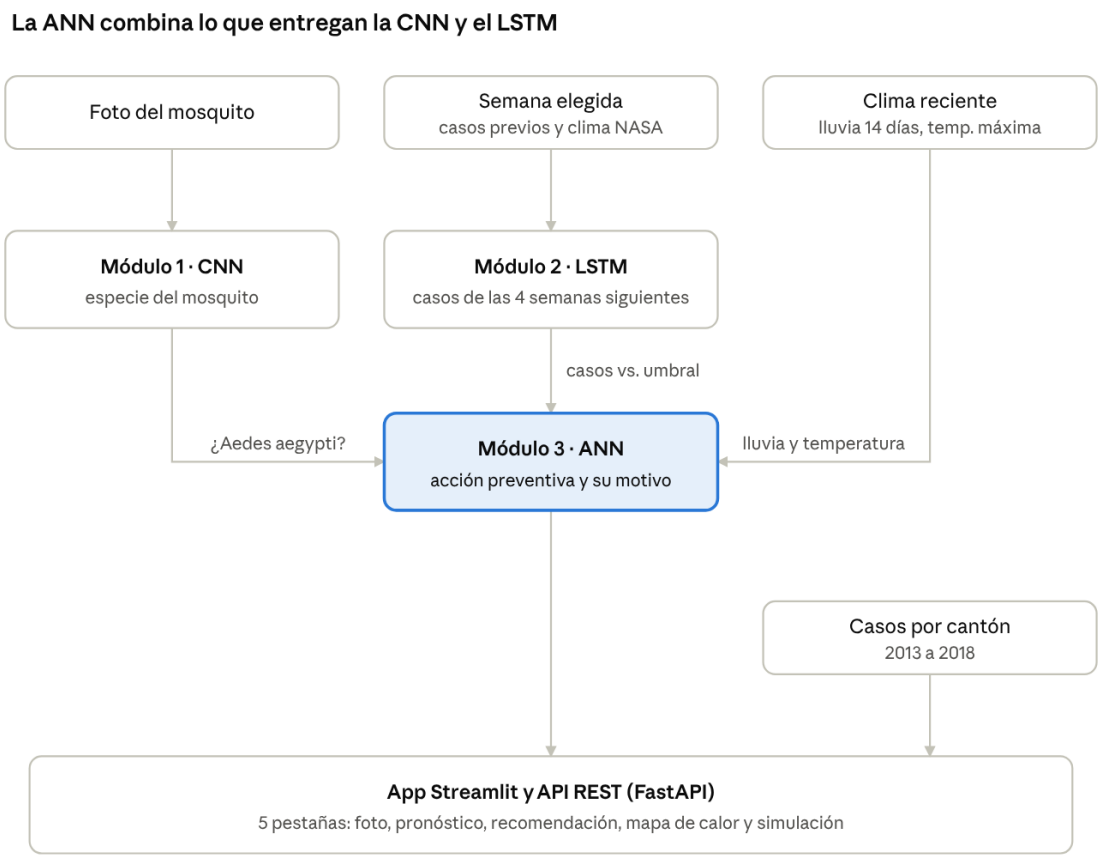
El clima de cada semana es el promedio de las 7 cabeceras de provincia, ponderado por población. Para 2025 y 2026 escribimos la clase ClimaNASAPower, que descarga el clima con el mismo método. Antes de usarlo, comprobamos que con ese método se obtiene el mismo clima de 2014 a 2024 que trae el dataset del proyecto.

Problemas de calidad que encontramos:

- **Subregistro de fin de año:** en todos los años, las semanas 51 a 53 reportan entre 20 % y 45 % menos casos de lo esperado, y las semanas 1 y 2 de enero cerca de 37 % más. Coincide con las vacaciones. Es un problema de registro, no de la enfermedad, así que dejamos los datos como están.
- **Datos cantonales que no cuadran:** la suma de los cantones es entre 1.3 y 1.7 veces mayor que el total nacional. Por ejemplo, en 2013 suman 81,942 casos, contra 49,993 del total nacional. Por eso usamos el dataset cantonal solo para comparar cantones entre sí, no como cifra oficial.
- **Lluvias extremas:** hay semanas con más de 480 mm de lluvia (el huracán Nate en 2017, Eta en 2020 y la tormenta Sara en 2024). Son reales, así que no las eliminamos. Para reducir su peso usamos el logaritmo de la lluvia.
- **Clases desbalanceadas:** en las fotos hay más *Aedes albopictus* (40.5 %) que *Culex* (27.8 %). En las acciones, el monitoreo normal es el 58.9 % de las filas y la fumigación focalizada solo el 7.1 %.

3. Diseño general del sistema

VectorIA tiene tres módulos que trabajan en cadena: lo que entregan la CNN y el LSTM es parte de la entrada de la ANN, y todo se muestra en la app.



Arquitectura de VectorIA · 3 módulos, app y API

La CNN dice si en la foto hay un *Aedes aegypti*. El LSTM pronostica los casos de las 4 semanas siguientes y los compara con el umbral endémico. La ANN junta esos dos resultados con la lluvia y la temperatura, y recomienda una acción. El código está organizado en clases, como pidió el profesor: PreparadorDatos y AnalizadorSerie para el EDA, PronosticadorDengue para el LSTM, RecomendadorAcciones para la ANN y ClimaNASAPower para descargar el clima.

4. Análisis exploratorio (EDA)

El EDA nos mostró que lo que más ayuda a predecir los casos de una semana son los casos de las semanas anteriores, y después la temperatura mínima con unas 6 a 9 semanas de rezago. El análisis está en notebooks/01_EDA.ipynb y usa las clases PreparadorDatos y AnalizadorSerie de src/data_prep.py.

Los hallazgos principales fueron:

- **Brotes y años tranquilos.** En 2023 y 2024 hubo más de 30,000 casos por año, unas seis veces más que en años tranquilos como 2017 o 2021 (unos 5,000). La serie no tiene una tendencia fija, sino ciclos de varios años.

- **Estacionalidad.** Casi todos los años tienen la misma forma: pocos casos en marzo y abril (unos 90 por semana) y más casos de junio a noviembre, con el máximo en agosto (unos 430 por semana). La lluvia empieza en mayo y los casos suben después.
- **Memoria de la serie.** Los casos de una semana se parecen muchísimo a los de la semana anterior (correlación de 0.97) y a los de 4 semanas antes (0.90). Esto es más fuerte que cualquier variable del clima.
- **Clima con rezago.** La temperatura mínima es la variable del clima más relacionada con los casos: su correlación sube de 0.44 sin rezago a 0.65 con 8 semanas. La lluvia en milímetros tiene una relación débil (0.26), pero con el logaritmo de la lluvia sube a 0.37 con 3 a 4 semanas de rezago.
- **Variables repetidas.** La temperatura máxima y la humedad tienen una correlación de -0.88, así que casi miden lo mismo.
- **Baseline.** Si se predice que cada semana tendrá los mismos casos que la misma semana del año anterior, en 2023 y 2024 el error es de 630.7 casos (RMSE). Ese es el número que el LSTM debía superar. En el módulo 2 sale 638.3, porque ahí se mide en las mismas semanas que pronostica el modelo.

Con estos resultados decidimos usar los casos en logaritmo, usar el logaritmo de la lluvia y darle al LSTM una ventana de 26 semanas. Esa ventana cubre tanto la memoria de los casos como los rezagos del clima.

5. Módulo 1: identificación de mosquitos (CNN)

El mejor modelo fue MobileNetV2, con un F1 macro de 0.82. La CNN propia llegó a 0.77. Ninguno alcanzó la meta del proyecto (0.85), pero los dos reconocen muy bien al *Aedes aegypti*, que es el vector principal del dengue.

Preparación de las imágenes. Las fotos tenían tamaños muy distintos, así que todas se redujeron a 224 × 224 píxeles en color. Usamos el 80 % para entrenar (3,562 imágenes) y el 20 % para evaluar (889). Durante el entrenamiento cada foto se modifica un poco al azar (rotación, desplazamiento, zoom y giro horizontal) para que la red no memorice las imágenes. También usamos *class weights* para compensar que hay menos fotos de *Culex*.

Modelos comparados:

1. **CNN propia (desde cero):** 4 bloques convolucionales de 32, 64, 128 y 256 filtros, cada uno con BatchNormalization y MaxPooling, y al final GlobalAveragePooling, una capa Dense de 128, Dropout de 0.5 y Softmax de 3 clases. Es la arquitectura que sugiere el proyecto con un bloque más, para captar detalles finos en las imágenes de 224 × 224. Se entrenó con Adam (tasa de aprendizaje 0.0003) y EarlyStopping, que la detuvo en la época 27.
2. **MobileNetV2 (transfer learning):** una red ya entrenada con ImageNet. Dejamos congelada su base y le agregamos nuestras propias capas finales.

Resultados con las 889 imágenes de prueba:

Métrica	CNN propia	MobileNetV2
Accuracy	0.76	0.82

Métrica	CNN propia	MobileNetV2
F1 macro	0.77	0.82
F1 *Aedes aegypti*	0.88	0.91
F1 *Aedes albopictus*	0.74	0.81
F1 *Culex*	0.69	0.74

La matriz de confusión muestra que el error más común es confundir *Aedes albopictus* con *Culex*: en la CNN propia, 87 fotos de *albopictus* se clasificaron como *Culex* y 59 al revés. En cambio, cuando el modelo dice "*Aedes aegypti*" casi siempre acierta (precisión de 0.95 en la CNN propia y 0.97 en MobileNetV2).

Una limitación del experimento es que el mismo 20 % de imágenes se usó para vigilar el entrenamiento y para la evaluación final. Por eso el resultado puede ser un poco más favorable que con fotos totalmente nuevas.

6. Módulo 2: pronóstico de casos (RNN/LSTM)

El LSTM pronostica las próximas 4 semanas con un error de 173 casos (RMSE), contra 638 del baseline estacional. Para la semana siguiente su precisión es de 88.3 %. El trabajo está en notebooks/03_LSTM_Casos.ipynb y en la clase PronosticadorDengue de src/train/lstm.py.

Entradas y salida. Con cada ventana de 26 semanas el modelo predice las 4 semanas siguientes. Las entradas son el logaritmo de los casos, la temperatura mínima, el logaritmo de la lluvia, la humedad, la temperatura máxima y la semana del año (en seno y coseno, para que la semana 52 quede cerca de la semana 1).

En lugar de predecir los casos directamente, el modelo predice **cuánto cambian** los casos (en logaritmo) respecto a la última semana conocida. Esto lo decidimos porque el máximo de entrenamiento era de 1,000 casos y en la prueba hay semanas con 1,749. Si el modelo predice los casos directamente, no puede salirse del rango que vio al entrenar y subestima los brotes. Si predice el cambio, sí puede seguirlos.

División temporal. Entrenamiento de 2014 a 2021 (418 semanas), validación en 2022 (52 semanas) y prueba en 2023 y 2024 (104 semanas). El escalador MinMaxScaler se ajustó solo con los datos de entrenamiento, para no usar información del futuro.

Modelos comparados:

1. **SimpleRNN (modelo base):** SimpleRNN de 32 neuronas y Dense de 4 salidas.
2. **LSTM (modelo principal):** LSTM de 64 neuronas, LSTM de 32, Dropout de 0.2 y Dense de 4 salidas.

Los dos usan activación ReLU, el optimizador Adam, la pérdida MSE y EarlyStopping con paciencia de 15 épocas, siguiendo el estilo del ejercicio de clase.

Resultados en la prueba (2023 y 2024):

Modelo	RMSE (casos)	MAE (casos)	Precisión del pronóstico
Baseline estacional	638.3	455.7	25.2 %
SimpleRNN	231.3	144.0	76.4 %
LSTM	**173.4**	**105.8**	**82.6 %**

La precisión del pronóstico la calculamos así:

Precisión = 100 × (1 − suma de los errores absolutos / suma de los casos reales)

Como es normal, el error sube mientras más lejos se predice:

Semana pronosticada	RMSE (casos)	Precisión
Semana 1	119.6	88.3 %
Semana 2	162.1	83.9 %
Semana 3	190.1	80.8 %
Semana 4	208.7	77.5 %

Comparación con el umbral endémico. El umbral endémico es el nivel a partir del cual los casos se consideran más altos de lo normal para esa época del año. Lo calculamos como el percentil 75 de la suma de casos de esas mismas 4 semanas en los 3 a 5 años anteriores. Nuestro cálculo coincide al 100 % con el umbral de vectoria_acciones.csv en las 412 semanas. En la prueba, el LSTM acierta en el 85.1 % de las semanas si los casos van a estar por debajo del umbral, sobre el umbral o en más del doble.

2025 y 2026. Los datos llegan a diciembre de 2024. Para 2025 y 2026 el modelo predice 4 semanas, las agrega a la ventana y vuelve a predecir, hasta cubrir 104 semanas. Siguiendo la recomendación del profesor, solo las primeras 4 semanas son un pronóstico y el resto son escenarios posibles, porque el error se va acumulando. Para esos escenarios usamos el clima real de NASA POWER. Para probar si sirve, repetimos el ejercicio como si estuviéramos a inicios de 2023. Con el clima real el error fue de 563.5 casos (RMSE), contra 668.9 con el clima promedio. Aun así, ningún escenario anticipó el brote de 2023.

7. Módulo 3: recomendación de acciones (ANN)

La ANN cumple la meta del proyecto: la precision y el recall de las cuatro acciones están en 0.85 o más, con una exactitud general de 0.94. El trabajo está en notebooks/04_ANN_Acciones.ipynb y en la clase RecomendadorAcciones de src/train/ann.py.

Qué recomienda. La red elige una de cuatro acciones, que siguen las reglas del proyecto en este orden de prioridad:

1. **Alerta sanitaria:** los casos pronosticados superan el doble del umbral endémico.
2. **Fumigación focalizada:** los casos superan el umbral y se detectó *Aedes aegypti*.
3. **Eliminación de criaderos:** la lluvia de 14 días supera 80 mm y la temperatura máxima pasa de 27 °C.
4. **Monitoreo normal:** en cualquier otro caso.

Entradas. Casos pronosticados en 4 semanas, umbral endémico, lluvia de 14 días, temperatura máxima y si se detectó *Aedes aegypti*. Agregamos una variable calculada, ratio_casos (casos pronosticados entre el umbral), porque las dos primeras reglas dependen de esa relación y a la red le cuesta aprender divisiones por sí sola.

Arquitectura. Una red densa con dos capas ocultas de 64 y 32 neuronas (ReLU) y una salida Softmax de 4 clases. Usa el optimizador Adam, la pérdida *sparse categorical crossentropy* y 150 épocas. Los datos se escalan con StandardScaler.

Evaluación. Cada semana tiene 5 escenarios casi iguales. Si se separan al azar por fila, el modelo vería en el entrenamiento casi las mismas filas que en la prueba. Por eso usamos validación cruzada de 5 partes **agrupada por semana** (GroupKFold): cada semana se predice una sola vez, con un modelo que nunca la vio.

Acción	Precision	Recall	F1	Filas
Alerta sanitaria	0.95	0.92	0.94	379
Eliminación de criaderos	0.90	0.85	0.88	321
Fumigación focalizada	0.91	0.85	0.88	147
Monitoreo normal	0.95	0.98	0.96	1,213

Las clases más difíciles son la fumigación y la eliminación de criaderos, que quedan justo en 0.85 de recall. También probamos un conjunto de 3 redes y dio casi lo mismo (exactitud de 0.94 y F1 macro de 0.92 contra 0.91), así que el modelo final es una sola red, entrenada con todos los datos.

Hay que tomar en cuenta que las etiquetas se generaron con reglas, con un 3 % de ruido al azar. Por eso la red aprende a reproducir esas reglas, no decisiones reales de las autoridades de salud.

8. Aplicación Streamlit y API REST

Los tres modelos se usan desde una aplicación web de 5 pestañas (app/Home.py) que obtiene sus resultados de una API REST (api/main.py). Si la API no está encendida, la app carga los modelos directamente, así que siempre funciona.

Pestañas de la aplicación:

Pestaña	Qué hace el usuario	Qué recibe
1. Identificación entomológica	Sube la foto de un mosquito	La especie, su porcentaje de confianza y si es vector del dengue
2. Pronóstico epidemiológico	Elige una semana entre 2014 y 2026	Los casos de las 4 semanas siguientes graficados contra el umbral endémico. En semanas de 2014 a 2024 también ve los casos reales
3. Recomendación sanitaria	Revisa el pronóstico, la lluvia, la temperatura y la	La acción recomendada, su nivel de riesgo y una explicación del motivo

Pestaña	Qué hace el usuario	Qué recibe
	especie	
4. Vigilancia cantonal	Filtra por año y provincia	Mapa de calor de casos por cantón y año, ranking de cantones y distribución por provincia
5. Simulación de escenarios	Mueve controles de casos, umbral, lluvia y temperatura	Cómo cambia la recomendación (“¿qué pasaría si…?”)

En todas las pestañas aparece fijo el aviso de que la herramienta apoya la vigilancia y no diagnostica dengue en personas.

Rutas de la API (FastAPI):

Ruta	Para qué sirve
POST /predict/cnn	Recibe una foto y devuelve la especie y las probabilidades
POST /predict/lstm	Recibe una fecha y devuelve el pronóstico de 4 semanas, el umbral y la situación
POST /predict/ann	Recibe el pronóstico, el clima y la especie, y devuelve la acción con su explicación
GET /data/semanas	Lista las semanas que se pueden consultar
GET /data/cantones	Devuelve los casos por cantón, con filtros de año y provincia
GET /health	Indica si los tres modelos están cargados

La documentación de la API se genera sola en http://127.0.0.1:8000/docs. Para correr el sistema se levanta la API con uvicorn api.main:app --reload y después la app con streamlit run app/Home.py.

Antes de entregar probamos las cinco pestañas con la API encendida. Corregimos un error por el que la gráfica del LSTM no mostraba los casos reales, agregamos el mapa de calor cantonal y completamos las librerías que faltaban en requirements.txt.

9. Resumen de métricas

Dos de los tres módulos cumplen la meta que pide el proyecto. La CNN quedó cerca, a 0.03 de la meta.

Módulo	Métrica que pide el proyecto	Meta	Mejor resultado	¿Cumple?
1 · CNN	F1 macro	≥ 0.85	0.82 (MobileNetV2)	No, por 0.03
2 · LSTM	RMSE y MAE contra el baseline estacional	Menor error que el baseline	RMSE 173.4 contra 638.3; MAE 105.8 contra 455.7	Sí

Módulo	Métrica que pide el proyecto	Meta	Mejor resultado	¿Cumple?
3 · ANN	Precision y recall por clase	≥ 0.85	Entre 0.85 y 0.98 en las cuatro acciones	Sí

En el módulo 2, además, el LSTM tiene una precisión del pronóstico de 88.3 % para la semana siguiente y de 82.6 % en promedio para las 4 semanas.

10. Limitaciones y recomendaciones

La limitación más importante es que el sistema no puede anticipar un brote nuevo con meses de anticipación. Sirve para avisar con unas pocas semanas.

Limitaciones:

- **CNN:** no llegó a la meta de 0.85 y confunde *Aedes albopictus* con *Culex*. Además, las mismas imágenes se usaron para validar y para evaluar.
- **LSTM:** el error sube mientras más lejos se predice. Los escenarios de 2025 y 2026 no anticipan brotes, porque los brotes dependen de factores que no están en los datos, como el serotipo del virus o las campañas de control. Solo hay 574 semanas y pocos brotes grandes para aprender.
- **Subregistro de fin de año:** produce errores en diciembre y enero, porque los casos se registran tarde.
- **ANN:** las etiquetas salen de reglas y no de decisiones reales del Ministerio de Salud. Además, en la app la lluvia y la temperatura se ingresan a mano, en lugar de tomarse del clima de la semana elegida.
- **Datos cantonales:** no cuadran con el total nacional y solo llegan a 2018.
- **Escala nacional:** el pronóstico es para todo el país, no por provincia o cantón.

Recomendaciones para mejorar el sistema:

1. **CNN:** hacer *fine-tuning* de MobileNetV2 (entrenar también sus últimas capas con una tasa de aprendizaje baja), conseguir más fotos de *Culex* y separar un conjunto de prueba aparte. También conviene que la app use MobileNetV2, que fue el mejor modelo.
2. **LSTM:** agregar variables que ayuden a anticipar brotes, como el serotipo circulante, y probar el pronóstico por provincia con datos reales.
3. **ANN:** entrenarla con decisiones reales de las autoridades de salud cuando existan, y probar una división temporal (2017 a 2022 contra 2023 y 2024).
4. **App:** tomar automáticamente la lluvia y la temperatura de la semana elegida, y actualizar los datos cantonales.
5. **Uso:** tomar las primeras 4 semanas como pronóstico y el resto solo como escenarios posibles.

11. Aviso ético y conclusiones

VectorIA es una herramienta de apoyo a la vigilancia epidemiológica y entomológica. **No diagnostica dengue en personas** y no reemplaza el criterio de los profesionales de salud. Este aviso aparece fijo en la aplicación. Las recomendaciones del módulo 3 son una guía para priorizar acciones, no una orden.

Conclusiones:

- Logramos unir tres tipos de redes neuronales en un solo sistema: la CNN identifica el mosquito, el LSTM pronostica los casos y la ANN recomienda la acción. Todo se puede usar desde una app web y una API.
- El LSTM fue el modelo más útil para la vigilancia: reduce el error del baseline estacional en más de 70 % y sigue los brotes de 2023 y 2024 con 4 semanas de anticipación.
- La ANN cumple la meta en todas las clases. La CNN reconoce muy bien al *Aedes aegypti*, pero todavía le cuesta separar *Aedes albopictus* de *Culex*.
- Las decisiones más importantes salieron del EDA: trabajar con logaritmos, usar los casos pasados como entrada, predecir el cambio de los casos y separar los datos por tiempo o por semana para no hacer trampa en la evaluación.
- Para usar este sistema en la vida real harían falta datos más recientes, pronósticos por región y validación con las autoridades de salud.